

algorithm.

There are a several different triangulation algorithms. Some are meant to have a quick runtime, others are meant to be simple, and still others are meant to yield triangulations that have specific desirable properties. What we'll cover here is called the *Bowyer-Watson algorithm*, and it's designed to take a set of points as its input and output a Delaunay triangulation.

A *Delaunay triangulation (DT)* aims to avoid narrow, sliver triangles. It tends to output triangles that are somewhere close to equilateral. Remember that equilateral triangles have relatively small circumcircles and sliver triangles have relatively large circumcircles. With that in mind, consider the technical definition of a DT: for a set of points, it is the set of triangles connecting all the points in which no point is inside the circumcircle of any of the triangles. The large circumcircles of sliver triangles would be very likely to encompass one or more of the other points in the set, so a rule stating that no point can be inside any circumcircle leads to relatively few sliver triangles. If this is unclear, don't fret—you'll see it visualized in the next section.

Incrementally Generating Delaunay Triangulations

Our eventual goal is to write a function that will take any set of points and output a full Delaunay triangulation. But let's start with something simple: we'll write a function that takes an existing DT of n points and also one point that we want to add to it, and outputs a DT of $n + 1$ points. This “Delaunay expanding” function will get us very close to being able to write a full DT function.

NOTE

The example and images in this section are courtesy of LeatherBee (<https://leatherbee.org/index.php/2018/10/06/terrain-generation-3-voronoi-diagrams/>).

First, suppose that we already have the DT of nine points shown in [Figure 7-10](#).

Now suppose we want to add a 10th point to our DT ([Figure 7-11](#)).